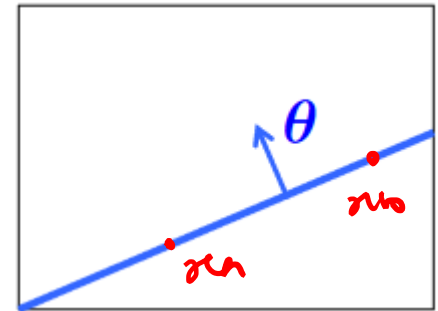


Linear Classification: The Perceptron

Linear Classifiers

- A hyperplane partitions $\mathbb{R}^d$ into two half-spaces, defined by the normal vector $\theta \in \mathbb{R}^d$
 - θ is orthogonal to any vector lying on the hyperplane
 - Assumed to pass through the origin as we have incorporated bias term θ_0 into it by $x_0 = 1$
- Consider classification with +1 and -1 labels



$$\theta^T x + b = 0$$

$$\theta^T x_a + b = 0$$

$$\theta^T x_b + b = 0$$

$$\theta^T (x_a - x_b) = 0$$

Linear Classifiers

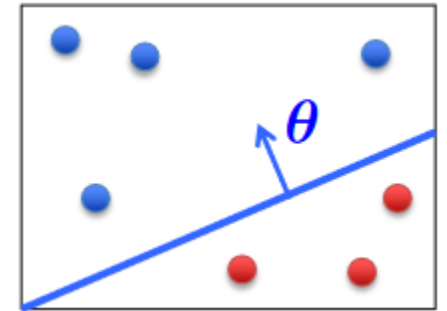
- Represent decision boundary by hyperplane

- $\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix} \quad \mathbf{x}^T = [1 \quad x_1 \quad \cdots \quad x_d]$

- $h(\mathbf{x}) = \text{sign}(\boldsymbol{\theta}^T \mathbf{x})$ where $\text{sign}(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ -1 & \text{if } z < 0 \end{cases}$

- $\boldsymbol{\theta}^T \mathbf{x} > 0 \implies y = +1$

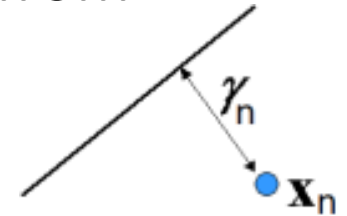
- $\boldsymbol{\theta}^T \mathbf{x} < 0 \implies y = -1$



Notion of Margins

- Geometric Margin $\gamma^{(i)}$ of an example $\mathbf{x}^{(i)}$ is its signed distance from hyperplane

$$\gamma^{(i)} = \frac{\boldsymbol{\theta}^T \mathbf{x}^{(i)}}{\|\boldsymbol{\theta}\|}$$



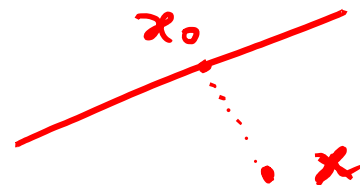
- $\gamma^{(i)}$ may be +ve or -ve based on which side of the plane $\mathbf{x}^{(i)}$ is
- Margin of a set $\{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(n)}\}$ with respect to $\boldsymbol{\theta}$ is the minimum absolute geometric margin

$$\gamma = \min_{1 \leq i \leq n} |\gamma^{(i)}| = \min_{1 \leq i \leq n} \frac{|\boldsymbol{\theta}^T \mathbf{x}^{(i)}|}{\|\boldsymbol{\theta}\|}$$

- Functional margin of $\boldsymbol{\theta}$ on a training example $(\mathbf{x}^{(i)}, y^{(i)})$: $y^{(i)}(\boldsymbol{\theta}^T \mathbf{x}^{(i)})$
 - +ve for correct prediction
 - ve for incorrect prediction

Length :-

$$\theta^T x + b = 0 \quad \text{--- Hyperplane}$$



$\frac{\theta}{\|\theta\|}$ = unit length normal vector

$(x - x_0) \perp \text{Hyperplane}$

$$x = x_0 + t \frac{\theta}{\|\theta\|}$$

$[t = \text{Signed Distance}]$

$\theta^T x_0 + b = 0$ $[\because x_0 \text{ lies on the hyperplane}]$

$$\theta^T x + b$$

$$= \theta^T \left(x_0 + t \frac{\theta}{\|\theta\|} \right) + b = \theta^T x_0 + t \frac{\theta^T \theta}{\|\theta\|} + b$$

$$= \underbrace{\theta^T x_0 + b}_{=0} + t \frac{\|\theta\|^2}{\|\theta\|} = t \|\theta\|$$

$$t = \frac{\theta^T x + b}{\|\theta\|} \quad [t = \gamma]$$

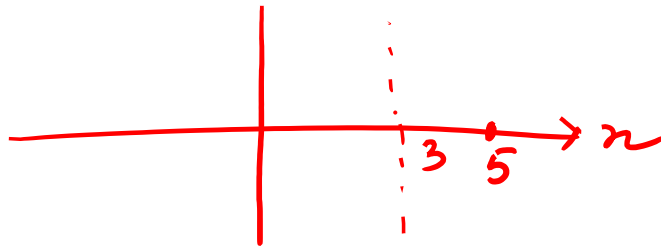
$$\boxed{\gamma^{(i)} = \frac{\theta^T x^{(i)} + b}{\|\theta\|}}$$

$$\gamma = t = \frac{0^T x + b}{\|a\|}$$

#

$$2x - 6 = 0$$

$$x = 3$$



$x = 5 \Rightarrow |5 - 3| = 2$ is distance x and the decision boundary.

$$\gamma = \frac{|2(5) - 6|}{2} = \frac{4}{2} = 2$$

$$20x - 60 = 0$$

$$x = 5 \Rightarrow 20(5) - 60 = 40 \text{ for value}$$

$$\gamma = \frac{20(5) - 60}{20} = 2$$

Loss function for linear classifier

- For a hyperplane based model

$$J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \max(0, -y^{(i)}(\boldsymbol{\theta}^T \mathbf{x}^{(i)}))$$

- If $y^{(i)}(\boldsymbol{\theta}^T \mathbf{x}^{(i)}) \geq 0$, then $\max(0, -y^{(i)}(\boldsymbol{\theta}^T \mathbf{x}^{(i)}))$ is zero for correct prediction
- Otherwise, $y^{(i)}(\boldsymbol{\theta}^T \mathbf{x}^{(i)}) < 0$ will make some positive loss for incorrect prediction
- Stochastic optimization on the loss yields $\frac{\partial J^{(i)}(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = -y^{(i)} \mathbf{x}^{(i)}$

The Perceptron

- $h(\mathbf{x}) = \text{sign}(\boldsymbol{\theta}^T \mathbf{x})$ where $\text{sign}(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ -1 & \text{if } z < 0 \end{cases}$

$$h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = -y^{(i)}$$

- The update rule for each training example $(\mathbf{x}^{(i)}, y^{(i)})$

$$\theta_j := \theta_j - \frac{\alpha}{2} (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

- For correct prediction, update will be stopped
 - For incorrect prediction update θ
- The update step can be re-written for incorrect prediction as

$$\theta_j := \theta_j + \alpha y^{(i)} x_j^{(i)}$$

- The perceptron rule: If $\mathbf{x}^{(i)}$ is misclassified, do $\boldsymbol{\theta} := \boldsymbol{\theta} + y^{(i)} \mathbf{x}^{(i)}$

Online Perceptron Algorithm

- Online learning – the learning mode where the model update is performed each time a single observation is received

Let $\boldsymbol{\theta} := [0, 0, \dots, 0]$

Repeat:

Receive training example $(\mathbf{x}^{(i)}, y^{(i)})$

If $y^{(i)}(\boldsymbol{\theta}^T \mathbf{x}^{(i)}) < 0$ //incorrect Prediction

$\boldsymbol{\theta} := \boldsymbol{\theta} + y^{(i)} \mathbf{x}^{(i)}$

Batch Learning

- The learning mode where the model update is performed after observing the entire training set

Given training data $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$

Let $\boldsymbol{\theta} := [0, 0, \dots, 0]$

Repeat:

Let $\Delta := [0, 0, \dots, 0]$

for $i = 1, \dots, n$, do

if $y^{(i)}(\boldsymbol{\theta}^T \mathbf{x}^{(i)}) < 0$

//incorrect Prediction

• $\Delta := \Delta + y^{(i)} \mathbf{x}^{(i)}$

• $\Delta := \frac{\Delta}{n}$

//compute average update

• $\boldsymbol{\theta} := \boldsymbol{\theta} + \alpha \Delta$

Until $\|\Delta\|_2 < \epsilon$

Why it works

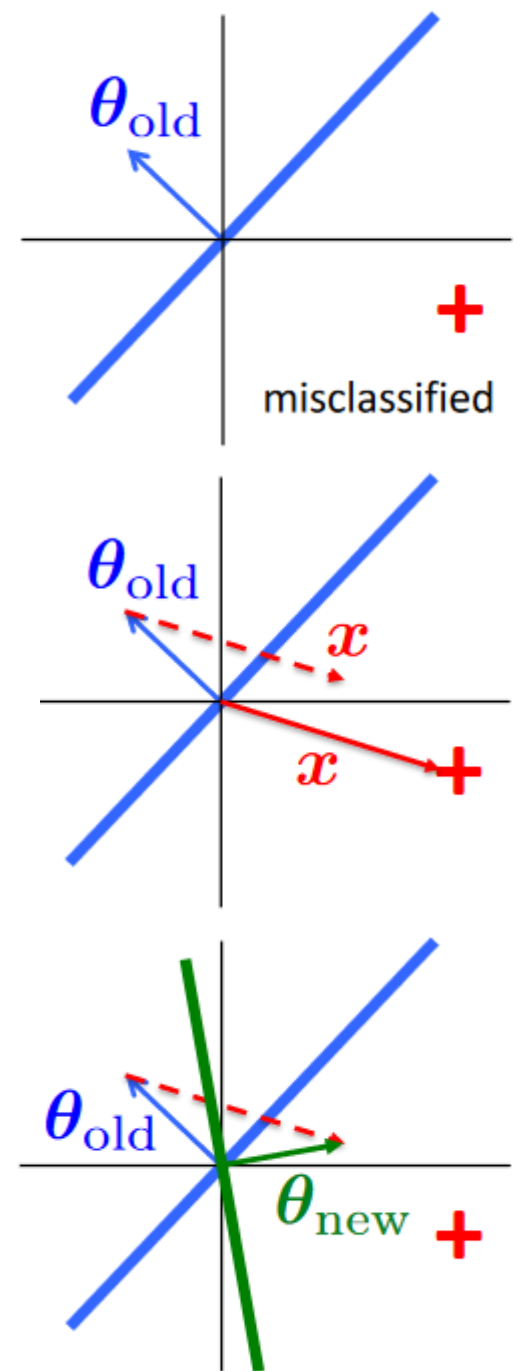
- Consider the misclassified positive example $y^{(i)} = +1$
- Perceptron wrongly predicts that $\theta_{old}^T \mathbf{x} < 0$
- Update step:

$$\theta_{new} = \theta_{old} + y\mathbf{x} = \theta_{old} + \mathbf{x} \quad (\because y = +1)$$

- Note

$$\begin{aligned}\theta_{new}^T \mathbf{x} &= (\theta_{old} + \mathbf{x})^T \mathbf{x} \\ &= \theta_{old}^T \mathbf{x} + \mathbf{x}^T \mathbf{x}\end{aligned}$$

- $\theta_{new}^T \mathbf{x}$ is less negative than $\theta_{old}^T \mathbf{x}$
- So we are progressing towards correct prediction.



Convergence of Perceptron

- **Theorem (Block & Novikoff):** If the training data is linearly separable with margin γ by a unit norm hyperplane $\boldsymbol{\theta}_*$ ($\|\boldsymbol{\theta}_*\| = 1$), then perceptron converges after $\frac{R^2}{\gamma^2}$ mistakes during training (assuming $\|\mathbf{x}\| \leq R$ for all $\mathbf{x}$)
- ⇒ • Guaranteed to find a separating hyperplane if one exists
- Convergence rate does not depend on the number of training examples or the data dimensionality.
- Depends only on the margin.

Effect of margin:- $\frac{R^2}{\gamma^2}$

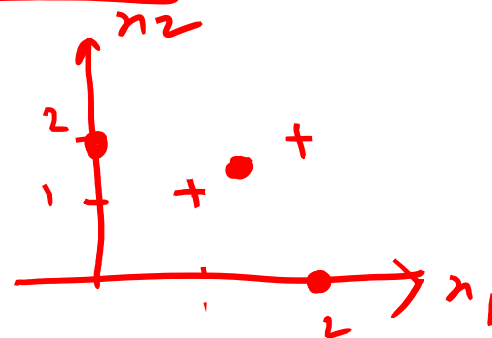
γ large $\longrightarrow$ Faster convergence

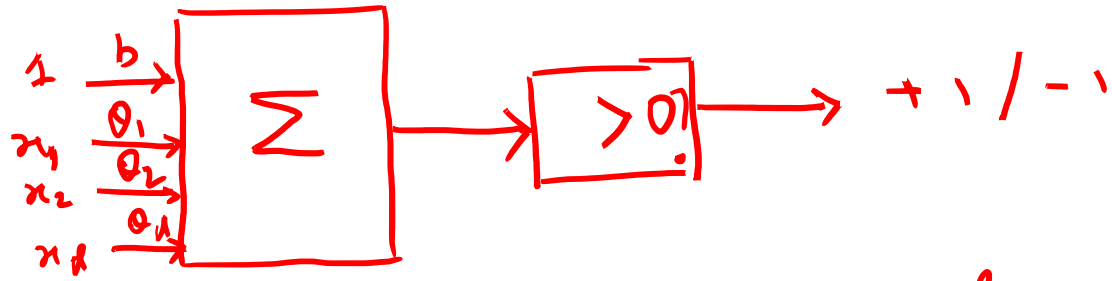
γ small $\longrightarrow$ Slower "

Effect of maximum Norm:- Large $R \longrightarrow$ Slower convergence
Small $R \longrightarrow$ Faster "

Non-linearly Separable Dataset:-

x_1	x_2	y
1	1	+1
2	2	+1
2	0	-1
0	2	-1
1.5	1.5	-1



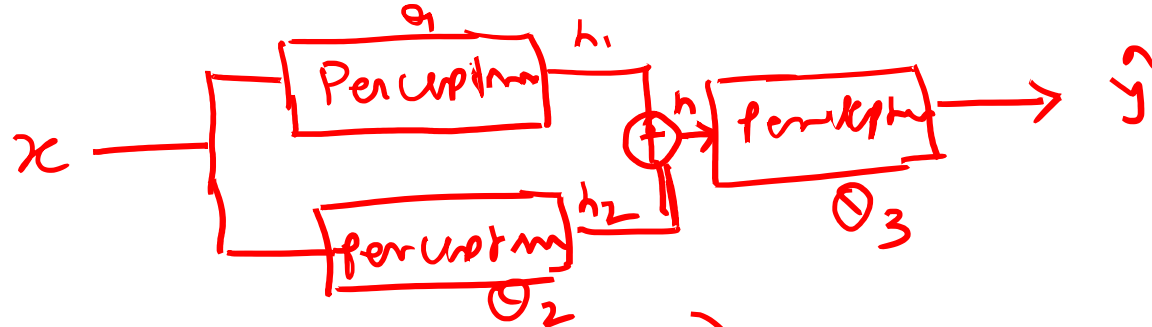


→ For linearly separable dataset - ~~the~~ perceptron will give us a classifier that ~~will~~ can classify the data perfectly.

→ For non-linearly separable dataset -
 — It makes mistake

→ update
 → Fixes for point
 → Do mistakes for other points

→ This cycle continues.

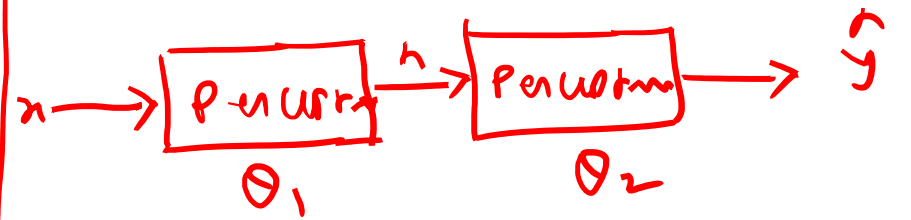


$$h_1 = \phi(\theta_1^T x + b_1)$$

$$h_2 = \phi(\theta_2^T x + b_2)$$

$$h = h_1 + h_2$$

$$\hat{y} = \phi(\theta_3^T h + b_3)$$



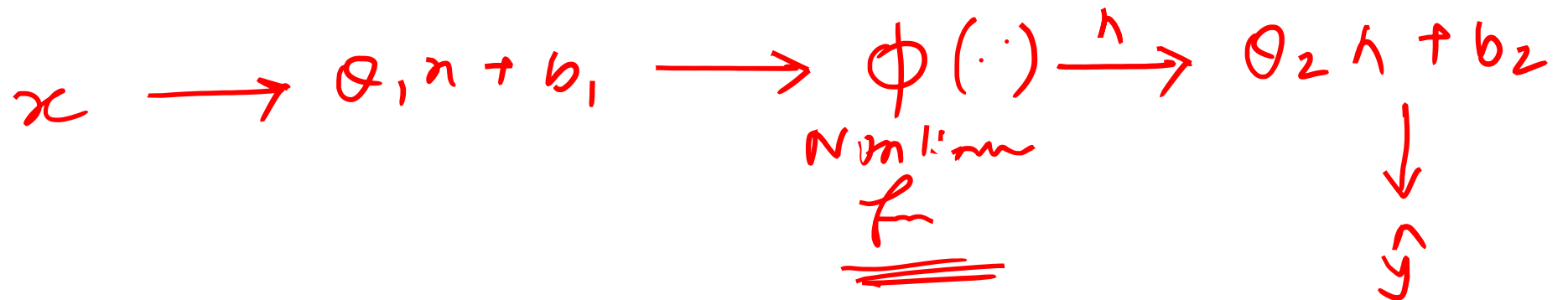
$$h = \theta_1 x + b_1$$

$$\hat{y} = \theta_2 h + b_2$$

$$\hat{y} = \theta_2(\theta_1 x + b_1) + b_2$$

$$= \underbrace{\theta_2 \theta_1}_{\theta} x + \underbrace{\theta_2 b_1 + b_2}_b$$

$$\hat{y} = \theta x + b$$



Naïve Bayes vs Perceptron

- Naïve Bayes predict classes based on the probability of the instance being that class
 - It learns $P(Y = y_k | X = \mathbf{x}_i)$
- Perceptron doesn't produce probability estimate
 - It estimates $\boldsymbol{\theta}$ from the training data
 - For new data find the sign of $\boldsymbol{\theta}^T \mathbf{x}^{(i)}$
 - Based on the sign assign a class